

CAPSTONE PROJECT

Employee Salary prediction using Machine learning Algorithms

Presented by: Sitesh Virju Gupta

College: Reena Mehta College

Organization: Edunet Foundation × IBM Skill Build AI/ML Internship Program

Duration :6 week

AICTE Student ID:STU67e55b65b2b2be1743084389

OUTLINE:

- **Problem Statement**
- **Learning Objective**
- **Methodology**
- **System Development Approach**
- **Data Visualization**
- **Model Comparison**
- **Deployment**
- **Result**
- **Conclusion**
- **Future Scope**
- **References**

Problem Statement:

- In today's job market, salary expectations are often unclear and vary widely across similar roles.
- Candidates struggle to estimate their worth based on experience, skills, and education.
- Employers face challenges in setting competitive and fair compensation packages.
- There's no simple tool for quickly predicting an expected salary range based on real-world data.
- This gap often leads to mismatches in job offers, candidate dissatisfaction, and negotiation breakdowns.

Learning Objective:

- To explore how data science and regression modeling can be applied to solve real-world challenges.
- To strengthen my Python development skills, especially in building GUI-based applications with interactive elements.
- To understand the end-to-end process of deploying a functional web app—from backend logic to live hosting.
- To gain hands-on experience in presenting technical work in a clear, professional format.
- To improve problem-solving through debugging, testing, and refining the application's performance.

Methodology:

- Began by identifying key salary-related features—like **years of experience**—and gathering reliable structured data for model training.
- Cleaned and preprocessed the dataset using pandas: **handled missing values, standardized formats, and encoded variables** where needed.
- Split the dataset into training and testing sets to validate accuracy and avoid overfitting.
- Visualized patterns with matplotlib to better understand feature influence.
- Compared the Models for the best model selection.
- Selected a classification algorithm (**Gradient Boosting Classifier**) to build a model that predicts salary category (**$\leq 50K$ or $> 50K$**) based on input attributes.
- Built an interactive GUI using **Streamlit** to allow user-friendly input and real-time predictions.
- Deployed the project to **Render**, making it accessible through a live link for testing and demonstration.

System Approach:

- **System Requirements**

- A machine with basic specs: **4GB RAM** or higher recommended for smooth execution
- Python 3. environment (**preferably 3.8+**)
- **Internet connection** for deployment and rendering the hosted app

- **Libraries Used**

- **pandas** – for structured data processing and manipulation
- **numpy** – for numerical operations supporting the model
- **Matplotlib** – for data visualization and chart plotting
- **scikit-learn** – to build and train the regression model
- **streamlit** – for creating the interactive and user-friendly web interface
- **joblib** – for saving and loading the trained model
- **render** – used as the hosting platform for deploying the live app

Data Visualization:

hours_per_week

- Most individuals work between 35–45 hours per week.
- A few outliers work extremely long hours, indicating workload imbalance.
- This variability helps assess productivity and earning potential.

educational_num

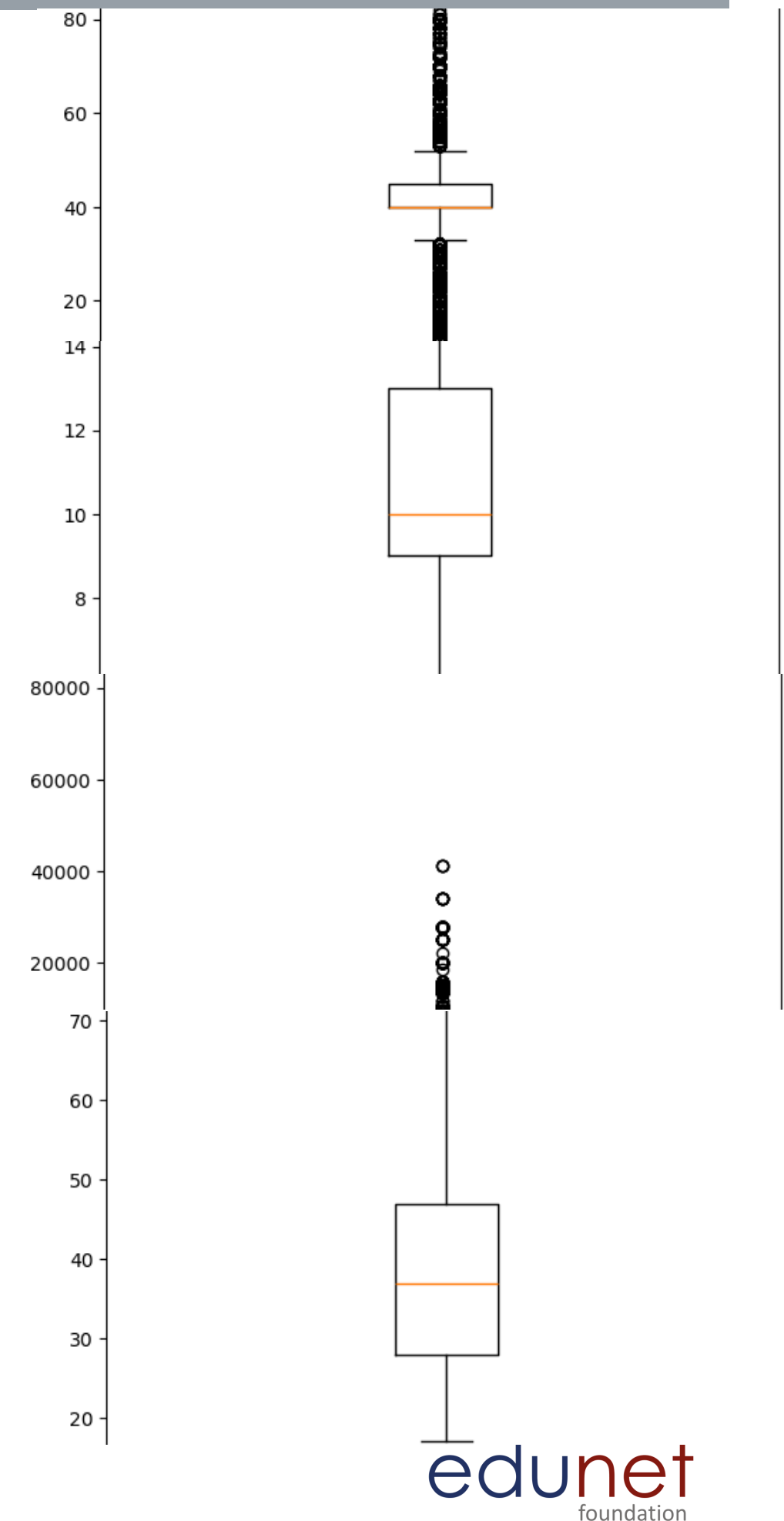
- Education levels are numerically encoded, typically ranging from 1 to 16.
- A noticeable concentration exists around mid-level education (e.g., high school, some college).
- Education is a strong predictor in salary modeling.

capital_gain

- The plot reveals a right-skewed distribution, with most people having zero or low gains.
- A small number of individuals report very high capital gains, possibly indicating investment income or asset growth.
- Highlights income disparity and its impact on overall salary patterns.

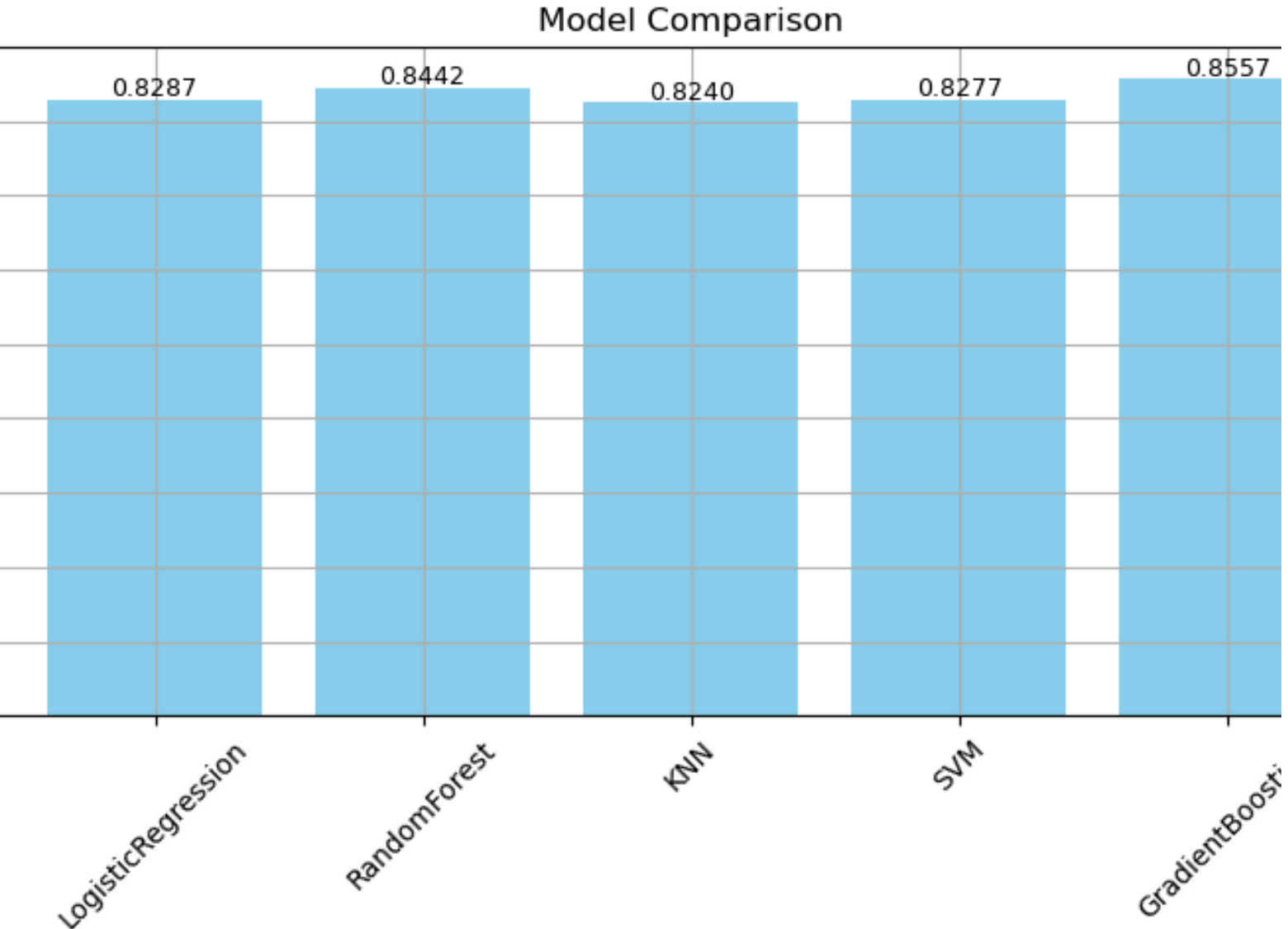
age

- Age distribution centers around 35–45 years, with a wider spread in the workforce.
- Younger and older workers show up less frequently but may have niche experience levels.
- Age ties directly to experience, which influences salary projections



Model Comparison:

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.8287	0.71	0.54	0.61
KNN	0.8240	0.67	0.59	0.63
SVM	0.8277	0.74	0.48	0.58
Random Forest	0.8442	0.72	0.62	0.66
Gradient Boosting	0.8557	0.77	0.61	0.68



- Observation:** Gradient Boosting delivered the most balanced and accurate predictions, particularly for classifying high-income cases (Class 1). It had the best overall F1-score and the highest accuracy.

Algorithm:

- Preprocessed the dataset using pandas, selecting relevant features including - **age, workclass, education/educational_num, marital_status, occupation, relationship, race, sex, capital_gain, capital_loss, hours_per_week, and native_country.**
 - These features reflect **individual demographics, employment type, education level, work habits, and financial variables**—all of which influence income patterns.
 - Preprocessed the data carefully using **techniques like label encoding, normalization, and missing value handling** to ensure **fairness and accuracy.**
- **Experimented with multiple models:**
 - - **Logistic Regression (Accuracy: 0.8287)**
 - - **KNN (Accuracy: 0.8240)**
 - - **SVM (Accuracy: 0.8277)**
 - - **Random Forest (Accuracy: 0.8442)**
 - - **Gradient Boosting (Accuracy: 0.8557)**
- Evaluated models using metrics like **Precision, Recall, and F1-Score** for both low-income and high-income classification (0 vs. 1).
- Chose **Gradient Boosting** for its best-balanced performance across all metrics—especially **strong recall and F1-score** on high-income class.
- Finalized **Gradient Boosting** as the core prediction engine for the app, ensuring reliable and nuanced salary estimation.

Deployment:

Model Saving

- Used joblib to serialize the trained Gradient Boosting Classifier, ensuring quick and efficient loading during app runtime.

App Interface Design

- Built a sleek, responsive frontend using Streamlit, allowing users to enter attributes like age, education, and hours/week to get salary predictions instantly.

Model Integration

- Connected the saved model to the Streamlit app so it could process real-time inputs and generate classification outputs on demand.

Testing & Debugging

- Ran multiple input combinations to validate the logic, test prediction accuracy, and ensure the app responded smoothly across use cases.

Live Hosting on Render

- Deployed the final app using Render, an easy-to-use cloud platform.
- Tackled and resolved deployment issues like missing files, environment dependencies, and path errors to ensure stable access for users.

Result-1:

app.py

employee salary prediction.ipynb

requirements.txt U

em ployee salary prediction.ipynb > # In ports

Generate + Code + Markdown | Run All | Clear All Outputs | Outline ...

Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

Numerical columns to scale
numeric_columns = ['age', 'educational_num', 'capital_gain', 'capital_loss', 'hours_per_week']

Define models
models = {
 "LogisticRegression": LogisticRegression(max_iter=1000),
 "RandomForest": RandomForestClassifier(),
 "KNN": KNeighborsClassifier(),
 "SVM": SVC(),
 "GradientBoosting": GradientBoostingClassifier()
}

Model comparison
results = {}

for name, model in models.items():
 preprocessor = ColumnTransformer([
 ('num', StandardScaler(), numeric_columns)
], remainder='passthrough')

 pipe = Pipeline([
 ('preprocess', preprocessor),
 ('model', model)
])

 pipe.fit(X_train, y_train)
 y_pred = pipe.predict(X_test)
 acc = accuracy_score(y_test, y_pred)
 results[name] = acc

print(f"\n {name} Accuracy: {acc:.4f}")
print(classification_report(y_test, y_pred))

LogisticRegression Accuracy: 0.8287

precision recall f1-score support

0 0.86 0.93 0.89 7819

1 0.71 0.54 0.61 2334

accuracy 0.83 9344

macro avg 0.78 0.73 0.75 9344

weighted avg 0.82 0.83 0.82 9344

RandomForest Accuracy: 0.8442

precision recall f1-score support

0 0.88 0.92 0.90 7819

1 0.72 0.62 0.66 2334

accuracy 0.84 9344

macro avg 0.80 0.77 0.78 9344

weighted avg 0.84 0.84 0.84 9344

KNN Accuracy: 0.8398

precision recall f1-score support

0 0.86 0.93 0.89 7819

1 0.71 0.54 0.61 2334

accuracy 0.83 9344

macro avg 0.78 0.73 0.75 9344

weighted avg 0.82 0.83 0.82 9344

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

ORDER

adult.csv

app.py

best_model.pkl

employee salary p...

requirements... U

app.py > ...

1 import streamlit as st

2 import pandas as pd

3 import joblib

4

5 # Must be FIRST Streamlit command

6 st.set_page_config(page_title="Employee Salary Predictor", page_icon="🌱", layout="centered")

7

8 # Load Model

9 try:

10 model = joblib.load("best_model.pkl")

11 except Exception as e:

12 st.error(f"Model failed to load: {e}")

13 st.stop()

14

15 # App Header

16 st.markdown("""

17 <div style="display: flex; justify-content: space-between; align-items: center; padding: 5px 10px;">

18

19

20 </div>

21 <div style="text-align:center; margin-top: 10px;">

22 <h1 style="color:#4CAF50;">Employee Salary Prediction WebApp </h1>

23 <p>Enter your details to predict whether your income is likely to exceed ₹50,000/year.</p>

24 </div>

25 """, unsafe_allow_html=True)

26

27 # Label Mappings

28

29 education_map = {

30 "9th": 5,

31 "10th": 6,

32 "11th": 7,

33 "12th": 8,

34 "Completed high school (Grade 12)": 9,

35 "Attended college but did not complete a degree": 10,

36 "Associate's degree in a vocational program": 11,

37 "Associate's degree in an academic program": 12,

38 "Bachelors": 13,

39 "Masters": 14,

40 "Completed a professional degree like law (LLB), medicine (MBBS), or similar": 15,

41 "Doctorate": 16

42 }

43 workclass_map = {"Private": 3, "Self-emp-not-inc": 1, "Local-gov": 5, "State-gov": 4, "Federal-gov": 6, "Without-pay": 7, "Never-worked": 2, "Unk

44 marital_map = {"Married": 1, "Single": 2, "Divorced": 3, "Separated": 4, "Widowed": 0}

Ln 5, Col 36 Spaces: 4 UTF-8 CRLF {} Python Completions quota reached Py

GitHub Link: <https://github.com/Siteshgupta123/Employee-Salary-Prediction-Project>

edunet
foundation

Result-2:

Provide Your Information

Age

30

Workclass

Private

Education Level

Bachelors

Marital Status

Married

Relationship

Wife

Occupation

Tech-support

Race

White

Gender

Female

Male

Capital Gain

10000

Capital Loss

0

Hours Per Week

40

Native Country

Cambodia

edunet foundation

IBM

Employee Salary Prediction WebApp

Enter your details to predict whether your income is likely to exceed ₹50,000/year.

Preview Your Input

	age	workclass	educational_num	marital_status	occupation	relationship	race	gender	ca
	0	30	3	13	1	0	0	0	0

What Do These Categories Mean?

Predict Income

Input Summary

- Age: 30
- Workclass: Private
- Education Level: Bachelors
- Marital Status: Married
- Occupation: Tech-support
- Relationship: Wife
- Race: White
- Gender: Female
- Capital Gain: ₹10000
- Capital Loss: ₹0
- Hours Per Week: 40
- Native Country: Cambodia

Predicted Salary: More than ₹50,000

Congratulations! You're projected to earn more than ₹50,000 annually!

-> Powered by GradientBoosting, Scikit-Learn Pipelines & Streamlit

-> Inputs above are user-friendly — mapped to model's label-encoded categories automatically.

Provide Your Information

Age

30

Workclass

Private

Education Level

9th

Marital Status

Married

Relationship

Wife

Occupation

Tech-support

Race

White

Gender

Female

Male

Capital Gain

edunet foundation

Employee Salary Prediction WebApp

Enter your details to predict whether your income is likely to exceed ₹50,000/year.

Preview Your Input

	age	workclass	educational_num	marital_status	occupation	relationship	ra
	0	30	3	5	1	0	0

What Do These Categories Mean?

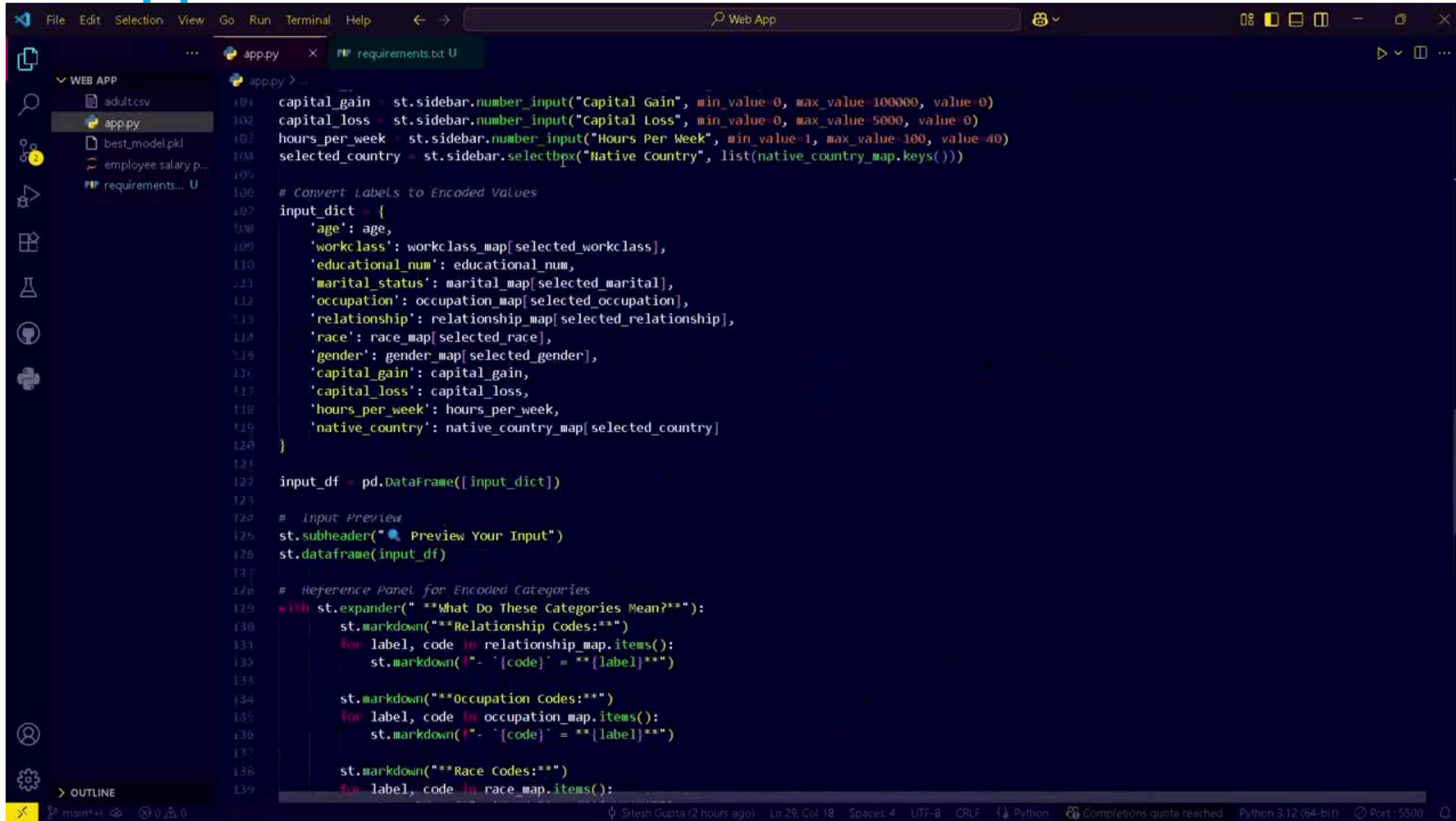
Predict Income

-> Powered by GradientBoosting, Scikit-learn Pipelines & Streamlit

-> Inputs above are user-friendly — mapped to model's label-encoded categories automatically.

Live Link:<https://employee-salary-prediction-project.onrender.com/>

Webapp Preview:



```
File Edit Selection View Go Run Terminal Help
Web App

WEB APP
  adult.csv
  app.py
  best_model.pkl
  employee_salary.p...
  requirements.txt

app.py
101 capital_gain = st.sidebar.number_input("Capital Gain", min_value=0, max_value=100000, value=0)
102 capital_loss = st.sidebar.number_input("Capital Loss", min_value=0, max_value=5000, value=0)
103 hours_per_week = st.sidebar.number_input("Hours Per Week", min_value=1, max_value=100, value=40)
104 selected_country = st.sidebar.selectbox("Native Country", list(native_country_map.keys()))
105
106 # Convert Labels to Encoded Values
107 input_dict = {
108     'age': age,
109     'workclass': workclass_map[selected_workclass],
110     'educational_num': educational_num,
111     'marital_status': marital_map[selected_marital],
112     'occupation': occupation_map[selected_occupation],
113     'relationship': relationship_map[selected_relationship],
114     'race': race_map[selected_race],
115     'gender': gender_map[selected_gender],
116     'capital_gain': capital_gain,
117     'capital_loss': capital_loss,
118     'hours_per_week': hours_per_week,
119     'native_country': native_country_map[selected_country]
120 }
121
122 input_df = pd.DataFrame([input_dict])
123
124 # Input Preview
125 st.subheader("🔍 Preview Your Input")
126 st.dataframe(input_df)
127
128 # Reference Panel for Encoded Categories
129 with st.expander(" **What Do These Categories Mean?** "):
130     st.markdown(" **Relationship Codes:** ")
131     for label, code in relationship_map.items():
132         st.markdown(f"- '{code}' = **{label}**")
133
134     st.markdown(" **Occupation Codes:** ")
135     for label, code in occupation_map.items():
136         st.markdown(f"- '{code}' = **{label}**")
137
138     st.markdown(" **Race Codes:** ")
139     for label, code in race_map.items():
140         st.markdown(f"- '{code}' = **{label}**")
```

> OUTLINE

Sitesh Gupta (2 hours ago) Ln 29, Col 18 Spaces: 4 UTF-8 CRLF Python Completions quota reached Python 3.12 (64-bit) Port: 5500

Conclusion:

Evaluation Summary

- The **Gradient Boosting** Classifier delivered the **highest accuracy (0.8557)** among all models tested, proving to be an effective solution for salary class prediction.
- The combination of thoughtful feature selection and robust preprocessing ensured fairness and relevance across demographic and employment-related inputs.
- Real-time prediction via a **Streamlit** web interface made the solution practical and user-friendly, supporting easy accessibility.
- Deployment on **Render** brought the project to life, enabling public usage with minimal setup requirements.

Challenges Encountered

- Addressing **missing values and encoding categorical features** required meticulous preprocessing.
- Deployment involved solving environment compatibility issues, handling file path errors, and ensuring all dependencies were correctly configured.

Potential Improvements

- Incorporate more advanced feature engineering (e.g. interaction terms, polynomial features) for better model interpretability.
- Implement user feedback tracking in the app to refine prediction relevance.
- Experiment with ensemble methods or deep learning alternatives to push accuracy even further.
- Add more visual polish and authentication layers for secure and scalable usage.

Future scope:

- **Technical Enhancements**

- **Expand to Regression Models:** Add salary range prediction using regression techniques like **Ridge** or **XGBoost** to offer more granular insights beyond classification.
- **Hyperparameter Tuning:** Use tools like **GridSearchCV** to fine-tune model parameters and further boost performance and reliability.
- **Multi-Model Option in UI:** Allow users to compare predictions from different models (e.g., Logistic Regression, Random Forest) for greater transparency and experimentation.
- **Feature Engineering:** Explore new derived features or interaction terms to improve predictive power and interpretability.

- **System Expansions**

- **User Authentication:** Integrate login/logout features for personalized user sessions and secure data handling.
- **Dynamic Dashboards:** Add interactive graphs and live metrics using libraries like Plotly or Altair for richer data exploration.
- **Mobile Optimization:** Make the app responsive across devices to ensure accessibility on phones and tablets.
- **Global Dataset Inclusion:** Enhance diversity by incorporating global salary data across industries and regions for broader applicability.
- **Model Interpretability Tools:** Integrate SHAP or LIME to help users understand how each feature influences the prediction outcome.

References:

1. UCI Machine Learning Repository – Adult Dataset

- Source of the primary dataset used for salary classification: <https://archive.ics.uci.edu/ml/datasets/adult>

2. Gradient Boosting Classification Explained – Scikit-Learn Documentation

- Detailed guide on how the algorithm works and how to implement it.
<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html>

3. Streamlit Documentation – Building and Deploying Interactive Apps

- Official resource used to design and deploy the frontend interface : <https://docs.streamlit.io/>

4. Render Deployment Guide

- Reference for deploying your Streamlit application to the cloud : <https://render.com/docs/deploy-streamlit>

5. Pandas Documentation – Data Preprocessing Techniques

- Used for data cleaning, encoding, and transformation: <https://pandas.pydata.org/docs/>

6. scikit-learn: Machine Learning in Python

- Pedregosa et al., Journal of Machine Learning Research, 2011.
- Provides theoretical background and toolkits used in model comparison and evaluation.:
<https://jmlr.org/papers/v12/pedregosa11a.html>

THANK YOU

- This project was developed as part of my internship under **Edunet Foundation** in collaboration with **IBM SkillBuild**, where I served as an **AI/ML Intern**.
- The result—an intelligent, real-time **Employee Salary Prediction Web App** built using advanced machine learning algorithms—reflects my hands-on learning and dedication to meaningful problem solving.
- I would like to express my heartfelt gratitude to my mentors:
Dr. Nanthini Mohan Mam
Channabasava Yadav Sir
for their guidance, encouragement, and support throughout this internship.